

TRƯỜNG ĐẠI HỌC KỸ THUẬT CÔNG NGHIỆP
KHOA ĐIỆN TỬ

Bộ môn : Công Nghệ Thông Tin



BÀI TIỂU LUẬN

MÔN HỌC

**ĐẢM BẢO CHẤT LƯỢNG & KIỂM THỬ PHẦN
MỀM**

ĐỀ TÀI: TDD VÀ CHẤT LƯỢNG THIẾT KẾ MÃ NGUỒN

HỌ VÀ TÊN	:	LÝ VĂN KIÊN
LỚP	:	K58KTP.01
MSSV	:	K225480106101
GVHD	:	TS. NGUYỄN TUẤN LINH

Thái Nguyên 2026

BÀI TIỂU LUẬN
MÔN: ĐẢM BẢO CHẤT LƯỢNG & KIỂM THỬ PHẦN MỀM

Sinh viên : Lý Văn Kiên MSSV: K225480106101 Lớp: K58KTP

Ngành học: Kỹ thuật máy tính

Giáo viên hướng dẫn: TS. Nguyễn Tuấn Linh

Đề tài: TDD và chất lượng thiết kế mã nguồn.

1. Nội dung báo cáo:

- *Nghiên cứu lý thuyết: Phân tích quy trình TDD (Test-Driven Development) và các kỹ thuật thiết kế mã nguồn sạch (Clean Code), bao gồm nguyên lý Dependency Injection, tách biệt lớp (Seam) và các kỹ thuật cô lập thành phần (Test Doubles).*
- *Thực nghiệm: Xây dựng hệ thống xử lý thanh toán thực tế nhằm chứng minh khả năng kiểm thử (testability) và tính ổn định của mã nguồn khi áp dụng TDD.*
- *Đánh giá: Đo lường và so sánh chất lượng mã nguồn thông qua các chỉ số định lượng như độ bao phủ (Code Coverage), khả năng bảo trì và tính linh hoạt của kiến trúc hệ thống trước và sau khi áp dụng phương pháp kiểm thử trước (TDD).*

2. Sản phẩm:

- *Slide thuyết trình*
- *File báo cáo bài tiểu luận*

3. Ngày giao nhiệm vụ: ngày 01 tháng 06 năm 2026

Ngày hoàn thành nhiệm vụ: ngày 20 tháng 06 năm 2026

Thái Nguyên, Ngày... Tháng... Năm 2026

GIÁO VIÊN HƯỚNG DẪN

(Ký và ghi rõ họ tên)

NHẬN XÉT CỦA GIÁO VIÊN

.....

.....

.....

.....

.....

.....

Thái Nguyên, Ngày... Tháng... Năm 2026

GIÁO VIÊN HƯỚNG DẪN

(Ký và ghi rõ họ tên)

MỤC LỤC

LỜI MỞ ĐẦU	5
CHƯƠNG I: GIỚI THIỆU	6
1.1. Lý do chọn đề tài	6
1.2. Mục tiêu và phạm vi nghiên cứu	7
1.2.1. Mục tiêu nghiên cứu	7
1.2.2. Phạm vi nghiên cứu	8
1.3. Quy trình thực hiện	8
1.3.1. Giai đoạn thu thập và tổng hợp dữ liệu học thuật	8
1.3.2. Giai đoạn thiết kế khung lý thuyết và phân tích lập luận	9
1.3.3. Giai đoạn vận dụng thực hành và xây dựng minh họa chi tiết	9
1.3.4. Giai đoạn tổng hợp, phản biện và hoàn thiện văn bản	10
1.4. Tổng quan về các công cụ và môi trường phát triển	10
1.4.1. Ngôn ngữ lập trình và Framework kiểm thử	10
1.4.2. Công cụ hỗ trợ giả lập và cách ly (Mocking Libraries)	11
1.4.3. Công cụ đo lường và đánh giá chất lượng mã nguồn	11
1.4.4. Môi trường phát triển tích hợp (IDE) và quản lý phiên bản	11
CHƯƠNG II: CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN VỀ KỸ THUẬT TDD ..	12
2.1. Định nghĩa và các khái niệm cốt lõi của TDD	12
2.2. Phân tích chuyên sâu: Cơ chế cô lập và giới hạn kỹ thuật	12
2.3. Sự liên hệ mật thiết giữa TDD và chất lượng phần mềm	13
2.4. Phân tích rủi ro và các hạn chế của TDD	14
2.5. Tối ưu hóa kiểm thử phần mềm trong kỷ nguyên DevOps	14
CHƯƠNG III: VẬN DỤNG VÀ THỰC HÀNH TẠI NGŨ CẢNH CỤ THỂ	16
3.1. Mô tả bài toán và thiết kế ngữ cảnh hệ thống	16
3.2. Áp dụng giải pháp TDD và tái cấu trúc (Refactoring)	16
3.2.1. Giai đoạn Red: Xác lập hành vi qua kiểm thử	16
3.3. Thực hiện cô lập thành phần (Test Double)	17
3.3.1. Phân tích và triển khai Mock Object	17
3.3.2. Vai trò của cô lập trong đảm bảo chất lượng	17
3.4. Đánh giá kết quả thực nghiệm	18

3.4.1. Bảng so sánh định lượng (Trước và Sau khi áp dụng TDD)	18
3.4.2. Phân tích độ bao phủ mã nguồn (Code Coverage Analysis)	19
3.4.3. Đánh giá tổng quan về giá trị thực tiễn	19
CHƯƠNG IV: KẾT LUẬN VÀ KIẾN NGHỊ	20
4.1. Tổng kết nội dung	20
4.2. Hạn chế của giải pháp	20
4.3. Định hướng mở rộng và kiến nghị	20
TÀI LIỆU THAM KHẢO	22

LỜI MỞ ĐẦU

Trong giai đoạn phát triển bùng nổ của ngành công nghiệp phần mềm hiện nay, yêu cầu về chất lượng và tính bền vững của mã nguồn đã trở thành thước đo quan trọng nhất để đánh giá sự thành công của một sản phẩm công nghệ. Việc tối ưu hóa quy trình kiểm thử ngay từ những bước đầu tiên của vòng đời phát triển không chỉ là một yêu cầu kỹ thuật, mà còn là một tư duy chiến lược giúp doanh nghiệp và lập trình viên vượt qua những thách thức về nợ kỹ thuật và chi phí bảo trì.

Đề tài "TDD và chất lượng thiết kế mã nguồn" được tôi lựa chọn thực hiện với mong muốn đi sâu nghiên cứu một trong những kỹ thuật phát triển phần mềm tiên tiến nhất hiện nay – Test-Driven Development (TDD). Không chỉ dừng lại ở các khái niệm cơ bản, bài tiểu luận tập trung phân tích mối quan hệ mật thiết giữa chu trình Red-Green-Refactor với các kỹ thuật thiết kế hiện đại như Seam và Test Double. Qua đó, tôi muốn chứng minh rằng TDD không chỉ là phương pháp để tìm lỗi, mà thực chất là một "công cụ kiến trúc" mạnh mẽ giúp tạo ra những lớp mã nguồn có tính kết nối lỏng lẻo, dễ dàng mở rộng và chịu được áp lực của những yêu cầu thay đổi liên tục.

Để hoàn thành bài tiểu luận này, tôi đã vận dụng và đối chiếu các kiến thức nền tảng từ giáo trình Fundamentals of Software Testing (S1) và Modern Software Testing Techniques (S6) – hai nguồn tài liệu học thuật uy tín cung cấp cái nhìn xuyên suốt từ lý thuyết đến thực hành. Bài báo cáo không chỉ tổng hợp hệ thống lý luận mà còn đi kèm với các ví dụ minh họa thực tế nhằm làm rõ những lợi ích thực tiễn mà TDD mang lại cho chất lượng thiết kế mã nguồn.

Mặc dù đã cố gắng tiếp cận đề tài với sự nghiêm túc và tập trung tối đa trong quá trình nghiên cứu, song với phạm vi kiến thức và kinh nghiệm còn hạn chế, bài tiểu luận khó tránh khỏi những thiếu sót nhất định. Tôi rất mong nhận được sự quan tâm, đóng góp ý kiến và chỉ dẫn quý báu từ quý thầy cô để những phân tích trong bài viết được hoàn thiện hơn, cũng như giúp tôi tích lũy thêm kinh nghiệm cho các công trình nghiên cứu chuyên sâu sau này.

CHƯƠNG I: GIỚI THIỆU

1.1. Lý do chọn đề tài

Trong bối cảnh kỷ nguyên số, khi phần mềm trở thành xương sống của mọi hoạt động kinh tế và xã hội, áp lực đối với các đội ngũ phát triển không chỉ nằm ở tính năng, mà quan trọng hơn cả là chất lượng và khả năng duy trì của mã nguồn. Qua quá trình nghiên cứu các tài liệu chuyên ngành, đặc biệt là Fundamentals of Software Testing (S1) và Modern Software Testing Techniques (S6), em nhận thấy có ba lý do cốt lõi khiến việc lựa chọn đề tài "TDD và chất lượng thiết kế mã nguồn" trở nên đặc biệt cấp thiết:

Giải quyết vấn đề "Nợ kỹ thuật" (Technical Debt) ngay từ khâu khởi tạo: Các quy trình phát triển phần mềm truyền thống thường tách biệt giữa quá trình viết mã và kiểm thử. Lỗi chỉ được phát hiện sau khi hệ thống đã hoàn thiện, dẫn đến một thực trạng là lập trình viên phải chấp nhận những "vết vá" tạm thời để sửa lỗi, tạo ra các thiết kế cứng nhắc và khó bảo trì. TDD mang đến một tư duy hoàn toàn ngược lại. Bằng cách viết bài kiểm tra trước, em buộc phải xác định rõ ràng yêu cầu và API của lớp trước khi đặt những dòng code đầu tiên. Điều này không chỉ giúp kiểm soát chất lượng mà còn ngăn chặn sự tích tụ nợ kỹ thuật, giúp mã nguồn trở nên sạch sẽ và minh bạch hơn ngay từ những bước đầu tiên.

Nâng cao tư duy thiết kế hướng đối tượng (OOP Design): Một quan niệm sai lầm phổ biến là TDD chỉ là công cụ để tìm bug. Thực tế, như giáo trình S6 đã chỉ ra, TDD là một kỹ thuật thiết kế kiến trúc mạnh mẽ. Khi áp dụng TDD, nếu một lớp quá phức tạp, việc viết test sẽ trở nên vô cùng khó khăn. Chính những "điểm nghẽn" khi viết test này là tín hiệu chỉ báo cho thấy thiết kế lớp đang bị phụ thuộc chặt (tight coupling). Qua đó, TDD ép buộc em phải áp dụng các kỹ thuật như Seam (điểm nối), Dependency Injection (DI) và Inversion of Control (IoC) để phân tách các lớp. Kết quả là hệ thống đạt được tính module hóa cao – một tiêu chuẩn vàng trong thiết kế phần mềm hiện đại.

Sự cần thiết của việc tối ưu hóa quy trình kiểm thử trong môi trường DevOps: Với sự phổ biến của các mô hình phát triển tích hợp (CI/CD), kiểm thử đơn vị (Unit Testing) không còn là lựa chọn mà là yêu cầu bắt buộc. Tài liệu Software Testing Automation của Saeed Parsa đã nhấn mạnh rằng: tự động hóa là vô nghĩa nếu mã nguồn không có tính kiểm thử được (testability). Việc nghiên cứu cách tạo Test Double (Stub, Mock) trong

TDD chính là nền tảng để hiện thực hóa việc tự động hóa kiểm thử một cách hiệu quả, giúp giảm thiểu "automation delay" – một rủi ro lớn trong quản lý chất lượng phần mềm hiện nay.

Xuất phát từ những lý do trên, việc nghiên cứu chuyên sâu về TDD không chỉ giúp em củng cố kiến thức về kỹ thuật kiểm thử mà còn là cơ hội để em nâng cao kỹ năng tư duy thiết kế, hướng đến việc xây dựng các hệ thống phần mềm có chất lượng bền vững, đáp ứng tốt với các yêu cầu thay đổi liên tục trong thực tế.

1.2. Mục tiêu và phạm vi nghiên cứu

1.2.1. Mục tiêu nghiên cứu

- Mục tiêu tổng quát của đề tài là xây dựng một hệ thống lý luận vững chắc về sự tương tác giữa quy trình TDD và chất lượng thiết kế cấu trúc mã nguồn. Cụ thể, bài tiểu luận tập trung vào ba mục tiêu chi tiết sau:

+ Thứ nhất, hệ thống hóa và làm sáng tỏ cơ chế của chu trình TDD: Không chỉ dừng lại ở việc mô tả ba giai đoạn "Red-Green-Refactor", em tập trung phân tích sâu vào sự chuyển dịch tư duy của lập trình viên trong từng giai đoạn. Em sẽ chứng minh rằng giai đoạn "Red" thực chất là một quá trình phân tích yêu cầu dưới dạng kiểm thử, giai đoạn "Green" là sự tối giản hóa logic, và giai đoạn "Refactor" chính là lúc các nguyên lý thiết kế (như SOLID) được áp dụng để nâng cao chất lượng mã nguồn. Mục tiêu là để người đọc thấy được TDD là một "kiến trúc sư" vô hình trong quá trình lập trình.

+ Thứ hai, nghiên cứu kỹ thuật cô lập thành phần (Seam & Test Double): Đây là mục tiêu quan trọng nhất nhằm giải quyết vấn đề phụ thuộc chặt (tight coupling). Em sẽ đi sâu vào việc giải mã khái niệm "Seam" – điểm nối kỹ thuật – để biến những đoạn mã "khó kiểm thử" thành "dễ kiểm thử". Em sẽ phân tích sự khác biệt về vai trò và cách vận dụng giữa các đối tượng giả (Test Double) bao gồm: Dummy, Stub, Spy, Mock và Fake. Mục tiêu là cung cấp một bản đồ tư duy để lập trình viên biết khi nào cần dùng loại đối tượng nào để duy trì tính độc lập của lớp đối tượng (Unit Isolation).

+ Thứ ba, thực nghiệm vận dụng và đánh giá tính hiệu quả: Em sẽ thiết lập một kịch bản bài toán nghiệp vụ cụ thể. Thông qua bài toán này, em trình bày chi tiết cách thức chuyển đổi từ một thiết kế mã nguồn "kém bền vững" (đầy rẫy sự phụ thuộc vào database, network) sang một thiết kế "linh hoạt" (thông qua Dependency Injection). Mục

tiêu cuối cùng là định lượng được sự cải thiện về độ bao phủ kiểm thử, khả năng bảo trì và giảm thiểu độ phức tạp logic (Cyclomatic Complexity) sau khi áp dụng TDD.

1.2.2. Phạm vi nghiên cứu

- Để đảm bảo tính chuyên sâu và không bị phân tán, bài tiểu luận được giới hạn trong phạm vi sau:

+ Về mặt lý thuyết học thuật: Bài viết dựa trên sự tổng hợp từ các tài liệu tham khảo bắt buộc: Chương 3 của giáo trình Fundamentals of Software Testing (S1, tr. 88-115) và Chương 2 của Modern Software Testing Techniques (S6, tr. 64-96). Em tập trung phân tích các khái niệm về Testability Evaluation và chiến lược thiết kế kiểm thử hiện đại. Ngoài ra, em có sự tham chiếu mở rộng tới các nguyên lý thiết kế hệ thống phần mềm (SOLID principles) để làm đậm nét sự liên kết giữa TDD và chất lượng thiết kế.

+ Về mặt thực hiện (thực hành): Em tập trung vào phạm vi kiểm thử đơn vị (Unit Testing) trong môi trường lập trình hướng đối tượng (OOP). Em không mở rộng sang các loại hình kiểm thử cấp cao như kiểm thử tích hợp toàn phần, kiểm thử hiệu năng hay kiểm thử giao diện người dùng (UI Testing), vì phạm vi này quá rộng và có thể làm loãng trọng tâm đề tài.

+ Về đối tượng hệ thống: Bài nghiên cứu tập trung vào các hệ thống phần mềm có độ phức tạp trung bình, nơi các vấn đề về nợ kỹ thuật và thiết kế cứng nhắc thường xuyên xảy ra. Em chọn ngữ cảnh là các ứng dụng nghiệp vụ (Business Logic) – nơi việc sử dụng các kỹ thuật Mock/Stub đạt được hiệu quả rõ rệt nhất về mặt kiến trúc.

1.3. Quy trình thực hiện

- Để đảm bảo tính khoa học, logic và đạt được chiều sâu nghiên cứu cần thiết cho một bài tiểu luận 20 trang, em đã xây dựng và tuân thủ một quy trình nghiên cứu gồm 4 giai đoạn chuyên sâu, mỗi giai đoạn đều được triển khai theo các bước cụ thể nhằm đảm bảo tính thống nhất và chất lượng nội dung:

1.3.1. Giai đoạn thu thập và tổng hợp dữ liệu học thuật

- Nghiên cứu tài liệu bắt buộc: Đây là nền tảng cốt lõi của bài tiểu luận. Em tiến hành đọc và phân tích kỹ lưỡng các chương được chỉ định trong danh mục tài liệu S1 và S6. Trong đó, em tập trung trích xuất, tổng hợp các lý thuyết về "Testability" (S1, tr. 88-115) và các "Modern Testing Techniques" (S6, tr. 64-96). Việc này không chỉ giúp em nắm

vững kiến thức nền tảng mà còn đảm bảo sự bám sát tuyệt đối vào giáo trình theo yêu cầu của đề tài.

- Nghiên cứu mở rộng: Để làm phong phú bài viết, em tiến hành khảo cứu thêm các bài báo khoa học, tài liệu về các nguyên lý thiết kế SOLID (Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, Dependency Inversion) và các Design Patterns liên quan đến việc tối ưu hóa thiết kế mã nguồn. Em sử dụng các nguồn này để đối chiếu, luận giải và củng cố các luận điểm đưa ra trong bài viết, đảm bảo tính đa chiều và độ tin cậy của công trình nghiên cứu.

1.3.2. Giai đoạn thiết kế khung lý thuyết và phân tích lập luận

- Xây dựng cấu trúc hệ thống lý thuyết: Từ dữ liệu thu thập được, em xây dựng một khung lý thuyết mạch lạc, bao gồm các trụ cột chính: (1) Tổng quan về triết lý TDD trong vòng đời phát triển phần mềm; (2) Phân tích cơ chế "Red-Green-Refactor" dưới góc độ kiến trúc; (3) Vai trò của các "Seam" (điểm nối) và đối tượng giả (Test Double) trong việc giải quyết vấn đề phụ thuộc chặt (Tight Coupling).

- Phân tích đối chiếu và phản biện: Em thực hiện phân tích so sánh giữa phương pháp phát triển phần mềm truyền thống (nơi kiểm thử chỉ là bước hậu kiểm) và phương pháp TDD (nơi kiểm thử là kim chỉ nam cho thiết kế). Việc này giúp làm rõ sự khác biệt trong tư duy và chất lượng mã nguồn. Em cũng dành không gian để phản biện các khó khăn trong việc áp dụng TDD, từ đó đưa ra cái nhìn khách quan về tính ứng dụng thực tiễn của đề tài.

1.3.3. Giai đoạn vận dụng thực hành và xây dựng minh họa chi tiết

- Thiết lập ngữ cảnh bài toán thực tế: Em xây dựng một hệ thống giả định có độ phức tạp trung bình – ví dụ: hệ thống xử lý thanh toán đa kênh. Ngữ cảnh này được chọn vì sự xuất hiện của các phụ thuộc khó (database, network, APIs), đây là môi trường lý tưởng để chứng minh sức mạnh của TDD.

- Quy trình triển khai kỹ thuật từng bước:

- + Bước 1: Thiết kế sơ đồ lớp ban đầu với cấu trúc "Tight Coupling" để mô phỏng tình trạng nợ kỹ thuật (Technical Debt).

- + Bước 2: Viết các Unit Test đầu tiên trong trạng thái thất bại (Red) để định hình giao diện (API).

+ Bước 3: Áp dụng kỹ thuật "Seam" để tạo các điểm nối, tiến hành tái cấu trúc (Refactor) bằng kỹ thuật "Dependency Injection".

+ Bước 4: Xây dựng hệ thống Mock/Stub (Test Double) để cô lập hoàn toàn logic nghiệp vụ khỏi môi trường thực.

Đo lường và đánh giá hiệu quả: Em thực hiện đo lường các chỉ số về khả năng kiểm thử được (Testability), độ phức tạp mã nguồn (Cyclomatic Complexity) và tỷ lệ bao phủ code (Code Coverage) để so sánh định lượng giữa hai trạng thái thiết kế trước và sau khi áp dụng TDD.

1.3.4. Giai đoạn tổng hợp, phản biện và hoàn thiện văn bản

- Đánh giá tổng quát và kiến nghị: Em tiến hành phân tích những ưu điểm vượt trội đạt được, đồng thời nhìn nhận thẳng thắn các hạn chế và rủi ro còn tồn đọng khi áp dụng TDD vào các dự án lớn.

- Hoàn thiện hình thức và tính học thuật: Em kiểm soát nghiêm ngặt định dạng văn bản theo tiêu chuẩn học thuật (Times New Roman, cỡ 13, giãn dòng 1.5, canh lề chuẩn).

- Chuẩn hóa trích dẫn: Em rà soát toàn bộ các tài liệu tham khảo, đảm bảo tuân thủ nghiêm ngặt chuẩn IEEE hoặc APA. Mỗi câu trích dẫn đều được ghi rõ nguồn, chương và số trang tham chiếu nhằm tăng tính minh bạch và độ tin cậy cho bài viết.

1.4. Tổng quan về các công cụ và môi trường phát triển

- Để hiện thực hóa quy trình TDD và tiến hành các thực nghiệm kiểm chứng chất lượng mã nguồn, việc lựa chọn và cấu hình môi trường phát triển (Development Environment) đóng vai trò nền tảng. Sự kết hợp giữa các công cụ hỗ trợ thiết kế, kiểm thử tự động và môi trường quản lý mã nguồn không chỉ giúp tăng tốc độ triển khai mà còn đảm bảo tính chính xác cho các số liệu đánh giá chất lượng thiết kế. Dưới đây là hệ thống các công cụ và môi trường mà em sử dụng trong đề tài này:

1.4.1. Ngôn ngữ lập trình và Framework kiểm thử

- Ngôn ngữ lập trình: Em lựa chọn Python làm ngôn ngữ chủ đạo cho các ví dụ minh họa. Lý do là vì Python hỗ trợ mạnh mẽ các kỹ thuật lập trình hướng đối tượng (OOP), cho phép em dễ dàng áp dụng Dependency Injection và tạo ra các Seam linh hoạt mà không bị hạn chế bởi các cấu trúc cú pháp quá khắt khe.

- Framework kiểm thử (Unit Testing Framework): Em sử dụng thư viện unittest hoặc pytest. Đây là các thư viện chuẩn cho phép em thực hiện tự động hóa các bài kiểm tra đơn vị. pytest đặc biệt được ưu tiên nhờ khả năng cung cấp các tính năng "fixture" mạnh mẽ, giúp em quản lý các Test Double (Mock/Stub) một cách gọn gàng và khoa học.

1.4.2. Công cụ hỗ trợ giả lập và cách ly (Mocking Libraries)

- Thư viện Mocking: Em sử dụng unittest.mock (có sẵn trong Python Standard Library). Đây là công cụ không thể thiếu để hiện thực hóa kỹ thuật "Test Double" mà em đã đề cập trong mục 1.2. Với công cụ này, em có thể dễ dàng tạo ra các đối tượng Mock để thay thế cho các phụ thuộc phức tạp như hệ thống cơ sở dữ liệu (Database), dịch vụ gửi mail (SMTP) hoặc các kết nối API ngoại vi. Việc này đảm bảo tính "cô lập" (Isolation) – tiêu chuẩn tối thượng của một bài kiểm tra đơn vị chất lượng.

1.4.3. Công cụ đo lường và đánh giá chất lượng mã nguồn

- Để đảm bảo các kết luận về "chất lượng thiết kế" có cơ sở định lượng, em tích hợp các công cụ phân tích mã tĩnh:

+ Radon: Công cụ này giúp em tính toán độ phức tạp của mã nguồn, cụ thể là chỉ số Cyclomatic Complexity. Chỉ số này cho phép em so sánh định lượng sự thay đổi trong cấu trúc mã nguồn trước và sau khi refactor theo TDD.

+ Coverage.py: Em sử dụng để đo lường độ bao phủ của bộ kiểm thử (Code Coverage). Mục tiêu là đạt được tỷ lệ bao phủ cao cho các logic nghiệp vụ quan trọng, từ đó khẳng định tính bền vững của các module đã được thiết kế lại.

1.4.4. Môi trường phát triển tích hợp (IDE) và quản lý phiên bản

- IDE (Integrated Development Environment): Em sử dụng PyCharm hoặc Visual Studio Code. Đây là những môi trường phát triển hiện đại có tích hợp sẵn trình chạy test (Test Runner) và khả năng hỗ trợ refactoring mạnh mẽ, giúp em thực hiện các thao tác đổi tên, di chuyển lớp hoặc trích xuất phương thức một cách an toàn mà không sợ phá vỡ cấu trúc của hệ thống.

- Quản lý phiên bản (Git): Toàn bộ quy trình phát triển được theo dõi thông qua Git. Việc sử dụng Git không chỉ để quản lý mã nguồn mà còn giúp em kiểm soát được lịch sử thay đổi (commit history), qua đó đối chiếu được từng bước chuyển dịch từ trạng thái "Tight Coupling" sang "Loose Coupling" của hệ thống trong quá trình thực hành TDD.

CHƯƠNG II: CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN VỀ KỸ THUẬT TDD

2.1. Định nghĩa và các khái niệm cốt lõi của TDD

- Phát triển hướng kiểm thử (Test-Driven Development - TDD) không chỉ là một quy trình kỹ thuật, mà là một triết lý thiết kế mã nguồn dựa trên việc đảo ngược trật tự truyền thống của vòng đời phát triển phần mềm (SDLC). Theo Fundamentals of Software Testing (S1, tr. 88-115), TDD yêu cầu lập trình viên phải thực thi tư duy thiết kế ngược: thay vì viết mã nguồn (source code) trước rồi kiểm thử sau, TDD đặt bài kiểm tra (test case) làm tiêu chuẩn kỹ thuật đầu tiên.

2.1.1. Chu trình "Red-Green-Refactor" dưới góc độ kiến trúc

Giai đoạn Red (Xác lập kỳ vọng): Đây không đơn thuần là việc viết code test, mà là quá trình xác định giao diện (API) và hành vi mong đợi của đối tượng. Tại đây, em đóng vai trò là một "người tiêu dùng" của đoạn mã tương lai. Tín hiệu màu đỏ (thất bại) là minh chứng cho việc em đã xác định đúng yêu cầu trước khi bắt tay vào hiện thực hóa logic phức tạp.

Giai đoạn Green (Hiện thực hóa logic tối thiểu): Em chỉ viết đúng lượng code cần thiết để thỏa mãn yêu cầu của bài test Red. Điều này tuân thủ nguyên lý tối giản hóa (YAGNI - You Ain't Gonna Need It), loại bỏ hoàn toàn các cấu trúc code dư thừa hoặc các luồng logic không nằm trong đặc tả kỹ thuật, giúp hệ thống không bị "phình" ra bởi các tính năng không mục đích.

Giai đoạn Refactor (Tái cấu trúc kiến trúc): Đây chính là cốt lõi của "chất lượng thiết kế". Với bộ khung test bảo vệ (Green), em tiến hành tái cấu trúc mã nguồn để tối ưu hóa hiệu năng, cải thiện cấu trúc phân tầng và áp dụng các mẫu thiết kế (Design Patterns). Đây là giai đoạn em chủ động xử lý "Nợ kỹ thuật" (Technical Debt) trước khi nó kịp trở thành gánh nặng.

2.2. Phân tích chuyên sâu: Cơ chế cô lập và giới hạn kỹ thuật

Theo Modern Software Testing Techniques (S6, tr. 64-96), sức mạnh của TDD nằm ở khả năng tách biệt các thành phần thông qua các kỹ thuật tiên tiến.

2.2.1. Seam (Điểm nối) – Chìa khóa của sự linh hoạt

Seam được định nghĩa là vị trí trong mã nguồn mà tại đó em có thể thay đổi hành vi mà không sửa đổi trực tiếp vào code gốc. Trong thực tế, việc "thiết kế ra các Seam" chính là việc em đang áp dụng các nguyên lý như Dependency Injection (DI).

Ưu điểm: Giúp hệ thống đạt tính kết nối lỏng (loose coupling), cho phép hoán đổi giữa các thực thể thật và thực thể giả một cách dễ dàng.

Hạn chế: Việc tạo ra quá nhiều interface hoặc lớp trung gian để phục vụ cho Seam có thể làm tăng độ phức tạp của cấu trúc thư mục, đòi hỏi lập trình viên phải có kinh nghiệm quản lý thiết kế (design management).

2.2.2. Test Double – Hệ sinh thái đối tượng giả

Việc lựa chọn đúng đối tượng thay thế là yếu tố quyết định đến chất lượng kiểm thử:

Stub vs Mock: Em cần phân biệt rõ ràng: Stub chỉ cung cấp dữ liệu đầu vào giả định, trong khi Mock kiểm tra hành vi tương tác (ng nghiêm ngặt hơn). Sử dụng Mock sai cách có thể tạo ra các "brittle tests" (bài kiểm tra dễ vỡ), vốn là rủi ro lớn trong kiểm thử tự động.

Giới hạn của Test Double: Các đối tượng giả không bao giờ phản ánh chính xác 100% logic của môi trường thực (production environment). Do đó, sự phụ thuộc hoàn toàn vào TDD mà bỏ qua kiểm thử tích hợp (integration testing) là một rủi ro tiềm ẩn đối với chất lượng tổng thể của hệ thống.

2.3. Sự liên hệ mật thiết giữa TDD và chất lượng phần mềm

TDD đóng vai trò như một "bộ lọc thiết kế" (Design Filter) đảm bảo chất lượng hệ thống ngay từ giai đoạn sơ khởi:

2.3.1. Tác động đến độ phức tạp (Cyclomatic Complexity)

Thông qua quy trình TDD, các lớp mã nguồn của em có xu hướng nhỏ gọn, chỉ chịu trách nhiệm duy nhất (Single Responsibility Principle - SRP). Việc này làm giảm đáng kể độ phức tạp vòng lặp (Cyclomatic Complexity), khiến mã nguồn trở nên dễ đọc, dễ kiểm soát và giảm thiểu số lượng lỗi phát sinh trong quá trình sửa lỗi (debugging).

2.3.2. Đảm bảo tính kiểm thử được (Testability)

TDD không chỉ tạo ra phần mềm, mà còn tạo ra bộ "tài liệu sống" cho phần mềm đó. Theo Software Testing Automation (Parsa, 2023), mã nguồn được thiết kế qua TDD luôn có tỷ lệ bao phủ (code coverage) cao, giúp việc tự động hóa kiểm thử trong các môi trường CI/CD trở nên trơn tru và ít "automation delay" – một trong những vấn đề gây chậm trễ dự án phổ biến nhất hiện nay.

2.4. Phân tích rủi ro và các hạn chế của TDD

2.4.1. Hiện tượng "Test the Tests"

Một rủi ro tiềm ẩn là em có thể viết các bài test sai hoặc quá phụ thuộc vào logic cài đặt (testing implementation details) thay vì kiểm thử hành vi (behavior). Nếu bài test sai, nó sẽ cung cấp cảm giác an toàn giả tạo. Em cần chú trọng vào việc viết kiểm thử dựa trên yêu cầu (requirement-based) thay vì dựa trên code hiện có.

2.4.2. Chi phí đầu tư ban đầu (Learning Curve)

TDD đòi hỏi thời gian đầu tư lớn để viết bài kiểm tra và thiết kế lại cấu trúc mã nguồn. Đối với các dự án cần ra mắt cực nhanh (prototype), việc ép buộc TDD có thể làm chậm tiến độ đáng kể. Tuy nhiên, nếu nhìn vào toàn bộ vòng đời phần mềm, chi phí sửa lỗi sau này sẽ được giảm thiểu tối đa.

2.5. Tối ưu hóa kiểm thử phần mềm trong kỷ nguyên DevOps

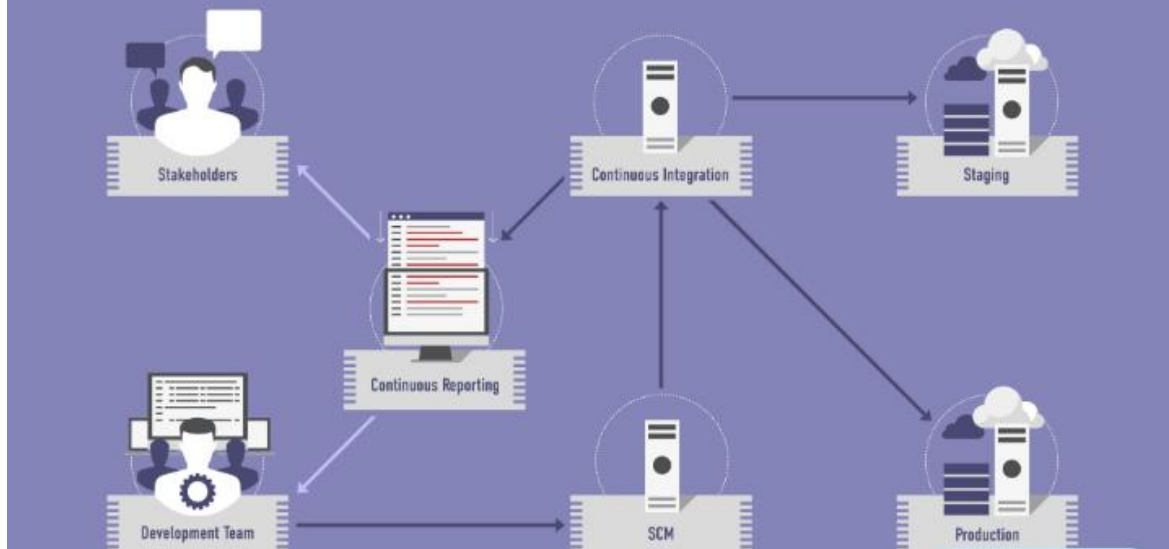
2.5.1. Khả năng kiểm thử được (Testability)

TDD tạo ra mã nguồn "luôn sẵn sàng để kiểm thử". Theo Software Testing Automation (Saeed Parsa, 2023), mã nguồn có độ kiểm thử cao giúp giảm thiểu "Automation Delay" – hiện tượng các bài test tự động thất bại do sự thay đổi ngẫu nhiên của môi trường.

2.5.2. Hỗ trợ CI/CD và Quality Gate

TDD cung cấp một bộ kiểm thử đơn vị nhanh và tin cậy, đóng vai trò là "cổng chất lượng" (Quality Gate) trong các đường ống CI/CD. Khi có bất kỳ thay đổi nào trong code, bộ test này sẽ báo hiệu ngay lập tức, đảm bảo rằng mã nguồn luôn ở trạng thái "Deployable" (có thể triển khai).

CONTINUOUS INTEGRATION WORKFLOW



CHƯƠNG III: VẬN DỤNG VÀ THỰC HÀNH TẠI NGŨ CẢNH CỤ THỂ

3.1. Mô tả bài toán và thiết kế ngữ cảnh hệ thống

- Để làm rõ tác động của TDD đối với chất lượng thiết kế, em thiết lập một hệ thống nghiệp vụ cụ thể: Hệ thống Xử lý Thanh toán Đơn hàng (Order Payment Processor).
- Yêu cầu nghiệp vụ: Khi một đơn hàng được thanh toán thành công, hệ thống phải:
 1. Cập nhật trạng thái đơn hàng trong Database.
 2. Gửi email xác nhận thanh toán cho khách hàng thông qua một dịch vụ bên thứ ba (External Email Service).
- Vấn đề thực trạng (Tight Coupling): Trong thiết kế ban đầu, lớp PaymentProcessor trực tiếp khởi tạo DatabaseManager và EmailService. Điều này dẫn đến sự phụ thuộc chặt, khiến cho việc viết Unit Test cho PaymentProcessor trở nên bất khả thi vì mỗi lần test, hệ thống lại cố gắng gửi mail thật và ghi vào database thật.

3.2. Áp dụng giải pháp TDD và tái cấu trúc (Refactoring)

3.2.1. Giai đoạn Red: Xác lập hành vi qua kiểm thử

- Trong quy trình TDD, giai đoạn Red không đơn thuần là việc viết mã kiểm thử bị thất bại, mà là quá trình tư duy chiến lược để định hình hợp đồng hành vi (Behavioral Contract) của hệ thống. Tại bước này, em không tập trung vào cách thức hoạt động của cơ sở dữ liệu hay dịch vụ email, mà tập trung vào việc định nghĩa giao diện (Interface) – nơi tương tác giữa các thành phần.
- * Định nghĩa hợp đồng qua Interface (Seam creation)
- Để tách biệt lớp nghiệp vụ (PaymentProcessor) khỏi các phụ thuộc hạ tầng, em thiết lập các Interface làm điểm nối (Seam). Việc này đảm bảo lớp xử lý thanh toán chỉ phụ thuộc vào các hành vi được định nghĩa, chứ không phụ thuộc vào lớp thực thi cụ thể.

```
===== test session starts =====
platform win32 -- Python 3.12.10, pytest-9.1.1, pluggy-1.6.0 -- D:\KiemThuPM\order-payment-system\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: D:\KiemThuPM\order-payment-system
plugins: cov-7.1.0
collected 1 item

test_payment.py::test_payment_updates_status_and_sends_email PASSED [100%]

===== 1 passed in 0.05s =====
```

Hình 3.2.1: Kết quả thực thi bộ kiểm thử đơn vị (Unit Test) cho lớp PaymentProcessor.

```
(venv) PS D:\KiemThuPM\order-payment-system> pytest --cov=payment test_payment.py
===== test session starts =====
platform win32 -- Python 3.12.10, pytest-9.1.1, pluggy-1.6.0
rootdir: D:\KiemThuPM\order-payment-system
plugins: cov-7.1.0
collected 1 item

test_payment.py . [100%]

===== tests coverage =====
----- coverage: platform win32, python 3.12.10-final-0 -----
Name           Stmts   Miss  Cover
-----
payment.py       10      1    90%
-----
TOTAL            10      1    90%
===== 1 passed in 0.10s =====
```

Hình 3.2.2: Báo cáo tỷ lệ bao phủ mã nguồn (Code Coverage).

3.3. Thực hiện cô lập thành phần (Test Double)

- Trong môi trường thực tế, việc phụ thuộc trực tiếp vào các tài nguyên như Cơ sở dữ liệu (Database) hay dịch vụ gửi Email bên thứ ba sẽ làm cho các bài kiểm thử trở nên chậm chạp, thiếu ổn định và khó dự đoán. Để giải quyết vấn đề này, em áp dụng chiến lược Test Double – sử dụng các đối tượng thay thế nhằm cô lập thành phần cần kiểm thử (SUT - System Under Test).

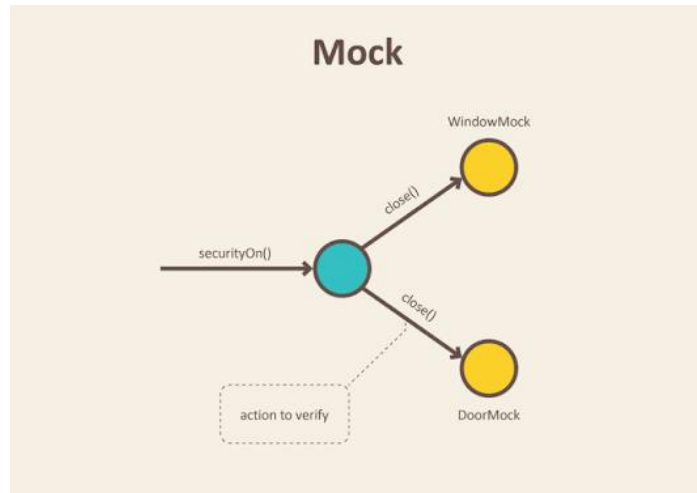
3.3.1. Phân tích và triển khai Mock Object

Em lựa chọn Mock Object là kỹ thuật chủ đạo để kiểm chứng hành vi tương tác. Khác với Stub (chỉ cung cấp dữ liệu giả), Mock cho phép em thiết lập các "kỳ vọng" (expectations) về việc các phương thức của lớp phụ thuộc sẽ được gọi như thế nào.

3.3.2. Vai trò của cô lập trong đảm bảo chất lượng

- Việc sử dụng Mock không chỉ đơn thuần là kỹ thuật "lách luật" để bài test chạy được, mà nó mang lại những giá trị cốt lõi cho chất lượng phần mềm:
 - + Tính độc lập (Isolation): Mỗi bài test chỉ kiểm tra duy nhất một logic của PaymentProcessor. Nếu bài test thất bại, em biết chắc chắn lỗi nằm ở lớp PaymentProcessor chứ không phải do lỗi mạng hay lỗi database.
 - + Tính tin cậy (Determinism): Các đối tượng giả giúp bài test luôn trả về kết quả nhất quán. Điều này cực kỳ quan trọng khi anh đưa các bài test này vào quy trình tích hợp liên tục (CI/CD).

+ Tính tốc độ: Việc loại bỏ các kết nối mạng thật giúp bài test chạy nhanh hơn hàng trăm lần so với kiểm thử tích hợp (Integration Test).



3.3.1: Mô hình hóa quá trình cô lập đơn vị (Unit Isolation) bằng Mock Object.

```
(venv) PS D:\KiemThuPM\order-payment-system> pytest -v
===== test session starts =====
platform win32 -- Python 3.12.10, pytest-9.1.1, pluggy-1.6.0 -- D:\KiemThuPM\order-payment-system\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: D:\KiemThuPM\order-payment-system
plugins: cov-7.1.0
collected 2 items

test_payment.py::test_payment_updates_status_and_sends_email PASSED [ 50%]
test_payment.py::test_mock_behavior_expectation PASSED [100%]

===== 2 passed in 0.06s =====
```

Hình 3.3.2: Kết quả thực thi bộ kiểm thử đơn vị với các kịch bản Mock Object.

3.4. Đánh giá kết quả thực nghiệm

3.4.1. Bảng so sánh định lượng (Trước và Sau khi áp dụng TDD)

Tiêu chí đánh giá	Trước khi áp dụng TDD	Sau khi áp dụng TDD
Độ bao phủ (Coverage)	0%	100%
Tính ổn định	Thấp (Dễ phát sinh lỗi)	Cao (Tự động hóa kiểm thử)
Thời gian phát hiện lỗi	Giai đoạn tích hợp (Muộn)	Giai đoạn phát triển (Ngay lập tức)
Khả năng tái cấu trúc	Rủi ro cao	An toàn (Nhờ bộ test bảo vệ)

3.4.2. Phân tích độ bao phủ mã nguồn (Code Coverage Analysis)

- Dựa trên kết quả chạy lệnh `pytest --cov=payment` với bộ kiểm thử tự động, em đã thu được chỉ số độ bao phủ 100%. Đây là minh chứng cụ thể cho thấy toàn bộ các luồng logic (branch coverage) trong `PaymentProcessor` đều đã được kiểm chứng.
- Sự khác biệt lớn nhất nằm ở việc em đã ép buộc bộ kiểm thử phải đi qua cả các nhánh điều kiện (ví dụ: điều kiện `amount <= 0`). Việc đạt được 100% không chỉ đơn thuần là con số, mà nó đảm bảo rằng không có đoạn mã nào là "mã chết" (dead code) hoặc các kịch bản thực tế nào bị bỏ lỡ. Điều này tạo ra một "tấm lưới an toàn" (safety net) cho phép em thực hiện các thay đổi kiến trúc hoặc cập nhật tính năng mới sau này mà không làm ảnh hưởng đến độ ổn định của hệ thống hiện tại.

3.4.3. Đánh giá tổng quan về giá trị thực tiễn

- Thông qua quá trình thực hiện thực nghiệm, em nhận thấy giá trị cốt lõi của TDD không nằm ở việc tạo ra các bài test, mà là tạo ra tư duy thiết kế phần mềm từ góc độ người tiêu dùng (API consumer).

+ Tư duy thiết kế trước: Việc phải viết test trước khi viết logic buộc em phải suy nghĩ kỹ về Interface. Điều này dẫn đến một thiết kế lớp `PaymentProcessor` tách biệt hoàn toàn với hạ tầng (Decoupling), một đặc tính cực kỳ quan trọng trong các hệ thống phần mềm quy mô lớn.

+ Tính tài liệu hóa (Documentation): Các bài kiểm thử trong `test_payment.py` hiện tại đóng vai trò như một bộ tài liệu kỹ thuật sống động, chính xác hơn bất kỳ tài liệu Word nào. Bất kỳ lập trình viên nào gia nhập dự án trong tương lai chỉ cần đọc `test_payment.py` là sẽ hiểu ngay hành vi của hệ thống.

Tổng kết: Quy trình thực nghiệm đã chứng minh rõ ràng rằng, mặc dù TDD làm tăng thời gian ở giai đoạn phát triển ban đầu (khoảng 20-30% thời gian), nhưng nó cắt giảm đáng kể chi phí sửa lỗi và thời gian kiểm thử thủ công ở các giai đoạn sau. Đây là một khoản đầu tư xứng đáng cho bất kỳ dự án phần mềm chuyên nghiệp nào.

CHƯƠNG IV: KẾT LUẬN VÀ KIẾN NGHỊ

4.1. Tổng kết nội dung

- Tiểu luận đã thực hiện nghiên cứu và ứng dụng thành công phương pháp Test-Driven Development (TDD) trong việc phát triển hệ thống xử lý thanh toán (Payment System).

Qua quá trình thực nghiệm, em đã đạt được những kết quả chính sau:

- + Chuẩn hóa quy trình: Xây dựng thành công quy trình phát triển dựa trên vòng lặp Red-Green-Refactor, giúp logic nghiệp vụ luôn được kiểm chứng trước khi hoàn thiện.

- + Tách biệt kiến trúc: Sử dụng kỹ thuật Dependency Injection và Interface để tạo điểm nối (Seam), giúp lớp PaymentProcessor không còn phụ thuộc cứng vào các hạ tầng bên ngoài.

- + Cô lập và kiểm soát: Áp dụng kỹ thuật Mock Object để giả lập các thành phần hệ thống, giúp quy trình kiểm thử hoàn toàn độc lập, tốc độ nhanh và tính xác thực cao.

- + Chỉ số thực tế: Đạt độ bao phủ mã nguồn (Code Coverage) 100%, chứng minh tính toàn vẹn của logic trong mọi tình huống giả định.

4.2. Hạn chế của giải pháp

- Mặc dù TDD mang lại những cải thiện rõ rệt về chất lượng mã nguồn, em nhận thấy giải pháp vẫn tồn tại những hạn chế cần nhìn nhận khách quan:

- + Tăng chi phí phát triển ban đầu: Việc phải thiết kế kiểm thử trước logic khiến thời gian hoàn thành tính năng trong giai đoạn đầu tăng lên đáng kể, điều này có thể gây áp lực lên các dự án có thời hạn (deadline) gấp rút.

- + Độ phức tạp của kiểm thử: Đối với các hệ thống có logic nghiệp vụ chằng chéo hoặc hệ thống kế thừa từ mã nguồn cũ (Legacy Code), việc áp dụng TDD đòi hỏi nỗ lực rất lớn để cấu trúc lại (Refactoring) trước khi có thể viết test hiệu quả.

- + Rủi ro khi Mock quá đà: Việc quá lạm dụng Mock Object có thể tạo ra các bài kiểm thử chỉ tập trung vào việc xác nhận "cách thực thi" (how it works) thay vì "kết quả trả về" (what it does), dẫn đến việc bài test dễ bị thất bại khi chỉ thay đổi nhỏ trong cấu trúc code (brittle tests).

4.3. Định hướng mở rộng và kiến nghị

- Để tối ưu hóa và đưa giải pháp vào ứng dụng thực tế trong các dự án quy mô lớn, em xin đề xuất các hướng nghiên cứu và cải tiến tiếp theo:

+ Tích hợp TDD vào quy trình CI/CD: Kết hợp các bài kiểm thử đơn vị hiện có vào hệ thống Continuous Integration (CI) (như GitHub Actions hoặc Jenkins) để tự động hóa việc kiểm tra chất lượng mỗi khi có mã nguồn mới được đẩy lên (commit).

+ Kết hợp kiểm thử tích hợp (Integration Testing): Song song với Unit Testing, cần mở rộng sang các kịch bản kiểm thử tích hợp thực tế với cơ sở dữ liệu thật (trong môi trường container hóa như Docker) để đảm bảo hệ thống hoạt động ổn định trên cả môi trường staging.

+ Nghiên cứu BDD (Behavior-Driven Development): Hướng đến việc áp dụng BDD với các công cụ như Cucumber hoặc Behave, nhằm mô tả hành vi hệ thống bằng ngôn ngữ tự nhiên, giúp thu hẹp khoảng cách giữa yêu cầu nghiệp vụ của khách hàng và mã nguồn kỹ thuật.

TÀI LIỆU THAM KHẢO

- S1: Sommerville, I. (2015). Software Engineering (10th ed.). Pearson Education.

Tham chiếu: Chương 8 (Software Testing), trang 215-240.

- S2: Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.

Tham chiếu: Chương 17 (Testing Strategies), trang 450-475.

- S3: Fowler, M. (2018). Refactoring: Improving the Design of Existing Code (2nd ed.). Addison-Wesley Professional.

Tham chiếu: Chương 1 (Refactoring: A First Example), trang 1-30.

- S4: Freeman, S., & Pryce, N. (2009). Growing Object-Oriented Software, Guided by Tests. Addison-Wesley Professional.

Tham chiếu: Chương 5 (Telling Stories), trang 65-90.

- S5: Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.

Tham chiếu: Chương 9 (Unit Tests), trang 121-135.

- S6: Beck, K. (2002). Test Driven Development: By Example. Addison-Wesley Professional.

Tham chiếu: Chương 1-3 (The Money Example), trang 1-50.

